Java加强

zcx

一、单元测试

- 针对最小的功能单元：方法，编写代码对其进行正确性测试
- **JUnit单元测试框架**：可以用来对方法进行测试，是第三方的开源框架
  - 优点：可以灵活的编写测试代码，可以针对某个方法执行测试，也支持一键完成对全部方法的自动化测试，且各自独立
  - 不需要程序员去分析测试的结果，会自动生成测试报告出来
- 使用步骤
  - 将JUnit.jar包导入到项目中(IDEA集成了这个框架，所以不需要导包了，但是还是要导的)
  - 为需要测试的业务类，定义对应的测试类，并为每个业务方法，编写对应的测试方法(必须：公共、无参、无返回值)
  - **测试方法上必须声明@Test注解**，然后在测试方法中，编写代码使用被测试的业务方法进行测试
  - **开始测试：选中测试方法，右键选择JUnit运行**，如果测试通过则是绿色，否则是红色
  -
    ```
    public class demo1 { 这是代码的两个方法，正在等待被编写测试类
    public static void printNumber(String name){
    System.out.println("名字长度是" + name.length());
    }
    public static int getMaxIndex(String data){
    if(data == null|| "".equals(data)){
    return -1;
    }
    return data.length();
    }
    }
    import org.junit.jupiter.api.Assertions;
    import org.junit.jupiter.api.Test; 这个导入的是JUnit5包
    public class demo1test { 测试类
    @Test 测试方法一
    public void testprintNumber(){
    demo1.printNumber("张三abc"); 自己编写测试样例
    demo1.printNumber("");
    demo1.printNumber(null);
    ```

```
}如果结果标红，表示代码有问题，测试失败
@Test
public void testgetMaxIndex(){ 测试方法二
int index1 = demo1.getMaxIndex("张三abc");
int index2 = demo1.getMaxIndex("");
int index3 = demo1.getMaxIndex(null);
System.out.println(index1);
System.out.println(index2);
System.out.println(index3);
Assertions.assertEquals(6, index1, "本轮测试失败");//故意写错的期待结果，如果预期结果和实际的输出结果不匹配，而代码没有错误，测试会标黄。
Assertions.assertEquals(-1, index2); 如果没有断言(Assertions.assertEquals)或者断言成功，则结果会是绿色，表示测试通过
Assertions.assertEquals(-1, index3);
}
}
```

可以在两个方法上任意地方右键，找到绿色的运行那个按键，就可以运行这个测试方法。当然可以对准一个类，运行这个类中的所有测试方法

二、反射(reflection)

- 反射就是：加载类，并允许以编程的方式解剖类中的各种成分(成员变量、成员方法、构造方法等)
- 比如，你声明一个对象，然后对象.就会出现一堆成员方法之类的选项，这就是反射，把类加载进程序并解剖类中的各种成分得到的，灵活的便捷的操作
- 反射第一步：获取class对象
  - 方式一：**Class c1 = 类名.class**
  - 方式二：调用Class提供方法：public static Class forName(String package);**Class c2 = Class.forName("包名.类名");**
  - 方式三：Object提供的方法：public Class getClass(); 需要类自己new一个对象，再**Class c3 = 对象.getClass();**
-
```
public class student { 这个是要反射的类
private String name;
private int age;
}
public class demo1 {
public static void main(String[] args) throws ClassNotFoundException { 这个是演示如何获取class对象
Class c1 = student.class; 方式一，直接类名.class
System.out.println(c1); //打印：class com.cxz.reflection.student
Class c2 = Class.forName("com.cxz.reflection.student"); 方式二，Class.forName(“包名.类名”)
```

```
System.out.println(c2);
student s = new student();
Class c3 = s.getClass(); 方式三，对象.getClass()
System.out.println(c3);
}
}
```

- 反射第二步：获取类的成分并操作
-

方法	作用
getConstructors()	获取所有公有构造方法
getDeclaredConstructors()	获取所有构造方法，包括私有的
getConstructor(Class...)	获取指定参数公有的构造方法
getDeclaredConstructor(Class...)	获取指定参数的构造方法，包括私有的
newInstance(Object...)	创建对象
setAccessible(true)	允许访问
getFields()	获取所有公有成员变量
getDeclaredFields()	获取所有成员变量，包括私有的
getField(String name)	获取指定公有成员变量
getDeclaredField(String name)	获取指定成员变量，包括私有的
set(Object obj, Object value)	为成员变量赋值
get(Object obj)	获取成员变量的值
setAccessible(true)	允许访问
getMethods()	获取所有公有成员方法
getDeclaredMethods()	获取所有成员方法，包括私有的
getMethod(String name, Class...)	获取指定公有成员方法
getDeclaredMethod(String name, Class...)	获取指定成员方法，包括私有的
invoke(Object obj, Object... args)	调用成员方法
setAccessible(true)	允许访问

- public class dog { **这个是dog类，为了测试反射，私有成员变量，公有私有构造器，公有私有成员方法**

```
private String name;
private int age;
private dog(){}
public dog(String name, int age){
this.name = name;
this.age = age;
}
private dog(String name){
this.name = name;
}
public String getName() {
return name;
}
public void setName(String name) {
this.name = name;
}
public int getAge() {
return age;
}
public void setAge(int age) {
this.age = age;
}
public void show(){
System.out.println("我的名字叫" + name + "，年龄是" + age);
}
private void show(String name){
System.out.println("我的名字叫" + name + "，年龄是" + age);
}
}
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.TestInstancePreConstructCallback;
import java.lang.reflect.Constructor;
import java.lang.reflect.Field;
import java.lang.reflect.InvocationTargetException;
import java.lang.reflect.Method;
public class demo2 { 这个是一堆测试类，主要测试获取class对象、获取构造器并对其操作、获取成员变量和对其操作、获取成员方法并对其操作
@Test 测试获取class对象
public void getclassInfo() {
Class c1 = dog.class;
```

```
System.out.println(c1.getName()); 获取完整类名，包括包名
System.out.println(c1.getSimpleName()); 获取单独的类名
}
@Test 测试获取构造器并对其操作
public void getclassInfo2() throws NoSuchMethodException, InvocationTargetException,
InstantiationException, IllegalAccessException {
Class c1 = dog.class;
Constructor[] cons = c1.getDeclaredConstructors();
for(Constructor c : cons){
System.out.println(c);
} 获取所有构造方法，包括私有
System.out.println("--------------------------------");
Constructor con2 = c1.getDeclaredConstructor(); 获取单个构造方法，包括private，看哪个是无参，就反射哪个
System.out.println(con2);
System.out.println("--------------------------------");
Constructor con3 = c1.getDeclaredConstructor(String.class); 获取单个构造方法，包括private，看哪个有参跟他对应，就反射哪个
System.out.println(con3);
System.out.println("--------------------------------");
Constructor con4 = c1.getDeclaredConstructor(String.class, int.class); 获取单个构造方法，包括private，看哪个有参跟他对应，就反射哪个
System.out.println(con4);
System.out.println("--------------------------------");
//获取构造器的作用依旧是创建对象
//可以暴力反射，暴力反射可以访问private属性和private方法
con2.setAccessible(true); //绕过访问权限，临时改变访问权限
dog d1 = (dog) con2.newInstance(); con2是私有构造器，需要暴力反射才能创建对象
System.out.println(d1);
System.out.println("--------------------------------");
dog d2 = (dog) con4.newInstance("张三", 18); //这和con4是public的，不需要暴力反射就能创建对象
System.out.println(d2);
}
@Test 测试获取成员变量并对其操作
public void getclassInfo3() throws NoSuchFieldException, IllegalAccessException {
Class c1 = dog.class;
Field[] fields = c1.getDeclaredFields();
for(Field f : fields){
System.out.println(f);
```

```
} 获取所有成员变量，包括私有
System.out.println("--------------------------------");
Field field = c1.getDeclaredField("name"); 获取单个成员变量，包括private，括号内是name，就反射这个变量
System.out.println(field);
System.out.println("--------------------------------");
Field field2 = c1.getDeclaredField("age"); 获取单个成员变量，包括private，括号内是age，就反射这个变量
System.out.println(field2);
System.out.println("--------------------------------");
//获取成员变量的目的依旧是取值赋值
dog d1 = new dog("ssad", 10); //要想对成员变量进行操作，需要先声明一个对象
//要么是暴力反射，把私有构造器临时改变，要么就是声明公有构造器
//上面就是声明了公有构造器
field.setAccessible(true); //对私有成员变量暴力反射
field.set(d1, "张三"); //赋值
System.out.println(d1.getName());
System.out.println("--------------------------------");
String name = (String) field.get(d1); //取值
System.out.println(name);
}
@Test 测试获取成员方法并对其操作
public void getclassInfo4() throws NoSuchMethodException, InvocationTargetException, IllegalAccessException {
Class c1 = dog.class;
Method[] methods = c1.getDeclaredMethods();
for(Method m : methods){
System.out.println(m);
} 获取所有成员方法，包括私有
System.out.println("--------------------------------");
Method method = c1.getDeclaredMethod("getAge"); 获取单个成员方法，包括private，括号内是getAge，就反射这个方法
System.out.println(method);
System.out.println("--------------------------------");
Method method2 = c1.getDeclaredMethod("setAge", int.class); 获取单个成员方法，包括private，括号内是setAge方法和int类型参数，就反射对应的这个带参方法
System.out.println(method2);
System.out.println("--------------------------------");
Method method3 = c1.getDeclaredMethod("show"); 获取单个成员方法，包括private，括号内是show方法，就反射这个方法
```

```
System.out.println(method3);
System.out.println("-------------------------------");
Method method4 = c1.getDeclaredMethod("show", String.class); 获取单个成员方法，包括private，括号内是show方法，带参数String，就反射这个带参方法
System.out.println(method4);
System.out.println("-------------------------------");
//方法的目的是调用和执行
//方法也需要用对象才能调，跟成员变量一样
dog d1 = new dog("张三", 18);
int age = (int) method.invoke(d1);这个方法是公有的，往上找method对象反射的是getAge方法，公有，有返回值，所以不需要暴力反射，而是这样写
System.out.println(age);
}
}
```

- **反射的作用、应用场景**
  - 基本作用：得到一个类的全部成分并操作
  - 可以破坏封装性
  - 可以绕过泛型的约束(比如你声明了一个String类型的集合，那么你正常add只能加String，但是用反射可以add任意对象，比如加double类型的等等)
  - **最重要的用途：适合做java的框架，基本上，主流的框架都会基于反射设计出一些通用的功能**
  - 如何用反射左一个简易版的框架(需求：对于任意一个对象，该框架都可以把对象的字段名和对应的值、保存到文件中去)
    - 定义一个方法，可以接受任意对象
    - **每收到一个对象后，使用反射获取该对象的Class对象，然后获取全部的成员变量**
    - 遍历成员变量，然后提取成员变量在对象中的具体值
    - 把成员变量名和其值，写出到文件中去即可，通过IO输出流实现

三、注解

- 是java代码里的特殊标记，比如@Override、@Test、@SuppressWarnings。作用是让其他程序根据注解信息来决定怎么执行该程序
- 注意：注解可以用在类上，构造器上，方法上，成员变量上，参数上，等位置
- **自定义注解**：自己定义注解。格式：public @interface 注解名{public 属性类型 属性名() default 默认值;}
- 特殊属性名：value。如果注解中只有一个属性(或者有其他属性，但其他属性均有默认值时)，那么使用注解时，value可以省略，直接写属性值即可
-
```
public @interface a { 注解a
String name();
int age() default 18; 注解a的属性age，有默认值18
String[] city();
```

```
}
public @interface b { 注解b
String value(); 注解b的属性value
int age() default 18; 注解b的属性age，有默认值18
}
@a(name = "java", city = {"上海"})
@b("去掉了value=，直接写属性值，就是这个字符串") 注解b虽然有别的属性，但是是有默认值的，所以可以当做只有value一个属性，使用注解b时可以省略value属性
public class test1 {
@a(name = "java", age = 12, city = {"上海"})
public static void main(String[] args) {
}
}注解注天注地注空气，哪都可以注解
```

- **注解的原理**：注解本质上就是一个接口，都是继承Annotation接口，注解的属性，实质都是抽象方法，@注解，就是在创建一个实现类对象，实现了该注解及Annotation接口。
- **元注解**：指的是注解注解的注解。白话：放在注解上面的注解。
- 元注解有4种：@Target、@Retention、@Documented、@Inherited，前两个最常用
- @Target：指定注解的作用目标,即声明被修饰的注解只能在哪些位置使用
-

作用目标	描述
TYPE	类、接口、枚举
FIELD	成员变量
METHOD	方法
CONSTRUCTOR	构造器
PARAMETER	方法参数
LOCAL_VARIABLE	局部变量

-

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Target;
@Target(ElementType.METHOD) 指定注解的作用目标，即声明被修饰的注解只能在方法上使用，在其他位置上使用会报错
public @interface mytest {
}
public class test2 {
private String name;
public test2(){} 如果在构造器上使用注解mytest，会报错，因为构造器不是方法，所以不能使用mytest注解，因为元注解规定了，如果想，就把ElementType.METHOD修改成ElementType.CONSTRUCTOR即可
```

```
public test2(String name){
this.name = name;
}
@mytest 方法上使用注解mytest，不会报错
public void show(){
System.out.println("show()方法执行了...");
}
}当然，@Target(ElementType.METHOD) 可以声明多个地方如
@Target({ElementType.METHOD, ElementType.TYPE})注解多个地方一次性，注意用逗号
隔开，并用花括号括起来
```

- @Retention：指定注解的保留周期
-

保留周期	描述
SOURCE	注解只保留在源文件中，编译时注解将被丢弃
CLASS	默认值。注解被保留在class文件中，但运行时注解将被丢弃
RUNTIME	开发常用，一般就用这个，因为单元测试时需要保留注解， 不然运行不是乱套了吗。注解被保留在class文件中，运行时注解将被保留

-

```
@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME) 指定注解的保留周期为运行时
public @interface mytest {
}
```

- **注解的解析**：判断类上、方法上、成员变量上是否存在注解，并把注解里的内容解析出来
- 如何解析注解？：要解析谁上的注解，就应该先拿到谁
  - 比如要解析类上面的注解，应该获取该类的Class对象，再通过着Class对象获取该类上的注解
  - 比如要解析方法上的注解，应该获取该方法对应的Method对象，再通过Method对象获取该方法上的注解
  - 比如要解析成员变量上的注解，应该获取该成员变量对应的Field对象，再通过Field对象获取该成员变量上的注解
  - Class、Method、Field对象都实现了AnnotatedElement接口，拥有解析注解的功能
  -

方法	描述
getDeclaredAnnotations()	获取当前对象上面的注解
getDeclaredAnnotation(Class< T > annotationClass)	获取指定的注解对象
isAnnotationPresent(Class< Annotation > annotationClass)	判断当前元素上是否有指定注解

- import java.lang.annotation.ElementType;
  import java.lang.annotation.Retention;
  import java.lang.annotation.RetentionPolicy;
  import java.lang.annotation.Target;
  @Target({ElementType.METHOD, ElementType.TYPE})
  @Retention(RetentionPolicy.RUNTIME)
  public @interface mytest2 { **注解mytest2,里面有三个属性**
  String value();
  double price() default 100;
  String[] city();
  }
  @mytest2(value= "aaaa",city={"vvvvvv","bbbbb"}) **类demo上面使用了注解mytest2**
  public class demo {
  @mytest2(value = "bbbbb",city = {"aaaaa","bbbbb"}) **方法go上面使用了注解mytest2**
  public void go(){
  }
  }
  import com.cxz.junit.demo1;
  import org.junit.jupiter.api.Test;
  import java.lang.reflect.Method;
  import java.util.Arrays;
  public class ademo2 {
  @Test **测试类，测试获取类demo上的注解mytest2的属性值**
  public void parseClass(){
  Class c1 = demo.class; **获取类demo的Class对象**
  if(c1.isAnnotationPresent(mytest2.class)){ **判断类demo上是否有注解mytest2**
  mytest2 mt = (mytest2) c1.getDeclaredAnnotation(mytest2.class); **获取类demo的注解mytest2对象**
  System.out.println(mt.value()); **获取类demo的注解mytest2的属性值**
  System.out.println(Arrays.toString(mt.city()));
  System.out.println(mt.price());
  System.out.println("--------------------------------");
  }
  }
  @Test **测试类，测试获取方法go上的注解mytest2的属性值**
  public void parseMethod(){
  Class c1 = demo.class; **获取类demo的Class对象**
  Method[] methods = c1.getDeclaredMethods(); **获取类demo的方法**
  for(Method m : methods){ **遍历方法**
  if(m.isAnnotationPresent(mytest2.class)){ **判断方法go上是否有注解mytest2**

```
mytest2 mt = m.getDeclaredAnnotation(mytest2.class); 获取方法go的注解mytest2对象
System.out.println(mt.value()); 获取方法go的注解mytest2的属性值
System.out.println(Arrays.toString(mt.city()));
System.out.println(mt.price());
System.out.println("--------------------------------");
}
}
}
}
```

- **注解的作用、应用场景**：可以开发出一个建议的junit框架。比如定义一个自定义注解Mytest，只能注解方法，存活范围一直都在，然后定义若干个方法，部分方法加上@Mytest，部分方法不加，那么就可以模拟一个JUnit程序，加了@Mytest的方法，就执行，不加的方法，就不执行，注解的属性可以完成方法的一些特殊要求，因为属性是可以被获取的。

四、动态代理

- 程序有的时候需要代理，如果对象嫌身上干了太多的活，就可以通过代理转移部分职责，不过对象有什么方法想要被代理，代理就一定要有对应的方法，代理本身相当于是把对象的方法中非核心的地方抽出来，再调用对象的核心的方法，
- 那么对象和代理之间需要中介来约束和实现，这个中介就是接口，对象实现方法，然后接口实现这些方法。
- 代理如何创建？：java.lang.reflect.Proxy类。提供了为对象产生代理对象的方法
-

方法	描述
newProxyInstance(ClassLoader loader, Class<?>[] interfaces, InvocationHandler h)	参数一：用于指定用哪个类加载器，去加载生成的代理类；参数二：指定接口，这些接口用于指定生成的代理长什么，也就是有哪些方法；参数三：用来指定生成的代理对象要干什么事情

-
```
public interface StarService { 定义一个接口，里面定义两个方法，这两个方法是要被代理的方法
void sing(String name);
String dance();
}
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
@Data
@AllArgsConstructor
@NoArgsConstructor
```

public class Star implements StarService{ **定义一个类，实现StarService接口，里面实现两个方法，这个类是要被代理的类，里面这个方法被重写了，实现的是核心的部分**

private String name;

@Override

public void sing(String singname) { **实现sing方法**

System.out.println(name + "正在唱歌，唱的是"+ singname);

}

@Override

public String dance() { **实现dance方法，这里要返回一个String=值**

System.out.println(name+"正在跳舞...");

return "跳完了哈哈哈哈";

}

}

import java.lang.reflect.InvocationHandler;

import java.lang.reflect.Method;

import java.lang.reflect.Proxy;

//专门负责创建代理对象并返回

public class ProxyUtil { **创建代理对象的加工厂**

public static StarService createProxy(Star s){ **创建代理对象，传入被代理对象，返回个接口类型的代理对象**

StarService proxy=(StarService) Proxy.newProxyInstance(

参数一：用于执行哪个类加载器去加载生成的代理对象

固定代码：类名.class.getClassLoader()

/

ProxyUtil.class.getClassLoader(),

/

参数二：代理对象需要实现的接口

固定代码：new Class[]{接口.class}

/

new Class[]{StarService.class},

/

参数三：代理类需要如何去代理(代理要做的事情)

固定代码：new InvocationHandler(){}

/

new InvocationHandler() {

@Override

public Object invoke(Object proxy, Method method, Object[] args) throws Throwable {

//声明代理对象要干的活

/

这个匿名内部类

```
参数一：proxy接收代理对象本身
参数二：method接收代理对象要调用的方法
参数三：args接收代理对象要调用方法时传递的参数
*/
if(method.getName().equals("sing")){
System.out.println("开始执行代理方法");
System.out.println("准备话筒，收钱20w");
}else if(method.getName().equals("dance")){
System.out.println("开始执行代理方法");
System.out.println("准备场地跳舞，收钱20w");
}
Object result = method.invoke(s, args);
return result;
}
});
return proxy;最后返回创建完的代理对象
}
}
import java.lang.reflect.Proxy;
public class demo1 { 主函数
//创建代理对象
public static void main(String[] args) {
Star star = new Star("cxz"); 声明被代理对象，传数据
StarService proxy = ProxyUtil.createProxy(star); 声明接口类型的代理对象，调用自己创建的代理对象加工厂，创建代理对象
proxy.sing("玛卡巴卡"); 代理对象调用两个方法就完事了
String dance = proxy.dance();
}
}
```

- **动态代理的好处、解决的实际问题**：比如有一万个方法，需要统计每个方法的性能分析，为进一步优化代码做铺垫。首先是有太多的重复代码，而且本身的代码模块本职工作不应该有性能分析，故可以用代理来解决